REVISA — Execution Pack de Artefatos

Este pack consolida os artefatos diretamente utilizáveis pela equipe em quatro blocos:

1. `openapi.yaml` consolidado
2. migrations Alembic por schema
3. seeds iniciais de roles, permissions e escopos
4. blueprint do monorepo com arquivos-base reais

---

1) openapi.yaml consolidado

```
openapi: 3.0.3
info:
  title: REVISA Platform API
  version: 1.0.0
  description: >-
    API consolidada da plataforma REVISA para gestão institucional,
    captação territorial, operação de polos, gabinete, dashboards,
    governança e privacidade.
servers:
  - url: https://api.revisa.local/api/v1
    description: Local
security:
  - bearerAuth: []
tags:
  - name: Auth
  - name: Users
  - name: Organizations
  - name: Vereadores
  - name: Teams
  - name: Persons
  - name: Consents
  - name: ContactsCapture
  - name: Polos
  - name: Demands
  - name: Tasks
  - name: Events
  - name: Dashboards
  - name: Reports
  - name: Geo
  - name: Audit
  - name: Privacy
paths:
  /auth/login:
    post:
      tags: [Auth]
```

```
    summary: Login do usuário
    security: []
    requestBody:
      required: true
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/LoginRequest'
    responses:
      '200':
        description: Login efetuado
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/TokenResponse'
      '401':
        description: Credenciais inválidas
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/ErrorResponse'
/auth/refresh:
  post:
    tags: [Auth]
    summary: Renova token de acesso
    security: []
    requestBody:
      required: true
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/RefreshRequest'
    responses:
      '200':
        description: Token renovado
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/TokenResponse'
/auth/logout:
  post:
    tags: [Auth]
    summary: Logout do usuário
    responses:
      '204':
        description: Logout efetuado
/auth/me:
  get:
    tags: [Auth]
    summary: Retorna o usuário autenticado
    responses:
```

```yaml
      '200':
        description: Usuário atual
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/User'

/users:
  get:
    tags: [Users]
    summary: Lista usuários
    parameters:
      - $ref: '#/components/parameters/Page'
      - $ref: '#/components/parameters/PageSize'
    responses:
      '200':
        description: Lista paginada
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/PaginatedUsersResponse'
  post:
    tags: [Users]
    summary: Cria usuário
    requestBody:
      required: true
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/UserCreateRequest'
    responses:
      '201':
        description: Usuário criado
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/User'

/users/{id}:
  get:
    tags: [Users]
    summary: Detalha usuário
    parameters:
      - $ref: '#/components/parameters/IdPath'
    responses:
      '200':
        description: Usuário encontrado
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/User'
```

```yaml
  patch:
    tags: [Users]
    summary: Atualiza usuário
    parameters:
      - $ref: '#/components/parameters/IdPath'
    requestBody:
      required: true
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/UserUpdateRequest'
    responses:
      '200':
        description: Usuário atualizado
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/User'

/users/{id}/roles:
  post:
    tags: [Users]
    summary: Vincula papéis ao usuário
    parameters:
      - $ref: '#/components/parameters/IdPath'
    requestBody:
      required: true
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/UserRolesAssignRequest'
    responses:
      '204':
        description: Papéis vinculados

/users/{id}/scopes:
  post:
    tags: [Users]
    summary: Vincula escopos ao usuário
    parameters:
      - $ref: '#/components/parameters/IdPath'
    requestBody:
      required: true
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/UserScopesAssignRequest'
    responses:
      '204':
        description: Escopos vinculados
```

```yaml
/organizations:
  get:
    tags: [Organizations]
    summary: Lista organizações
    responses:
      '200':
        description: Lista de organizações
        content:
          application/json:
            schema:
              type: array
              items:
                $ref: '#/components/schemas/Organization'
  post:
    tags: [Organizations]
    summary: Cria organização
    requestBody:
      required: true
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/OrganizationCreateRequest'
    responses:
      '201':
        description: Organização criada
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/Organization'

/organizations/{id}:
  get:
    tags: [Organizations]
    summary: Detalha organização
    parameters:
      - $ref: '#/components/parameters/IdPath'
    responses:
      '200':
        description: Organização
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/Organization'
  patch:
    tags: [Organizations]
    summary: Atualiza organização
    parameters:
      - $ref: '#/components/parameters/IdPath'
    requestBody:
      required: true
      content:
```

```
        application/json:
          schema:
            $ref: '#/components/schemas/OrganizationUpdateRequest'
    responses:
      '200':
        description: Organização atualizada
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/Organization'

/vereadores:
  get:
    tags: [Vereadores]
    summary: Lista vereadores
    responses:
      '200':
        description: Lista de vereadores
        content:
          application/json:
            schema:
              type: array
              items:
                $ref: '#/components/schemas/Vereador'
  post:
    tags: [Vereadores]
    summary: Cria vereador
    requestBody:
      required: true
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/VereadorCreateRequest'
    responses:
      '201':
        description: Vereador criado
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/Vereador'

/teams:
  get:
    tags: [Teams]
    summary: Lista equipes
    responses:
      '200':
        description: Lista de equipes
        content:
          application/json:
            schema:
```

```
              type: array
              items:
                $ref: '#/components/schemas/Team'
  post:
    tags: [Teams]
    summary: Cria equipe
    requestBody:
      required: true
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/TeamCreateRequest'
    responses:
      '201':
        description: Equipe criada
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/Team'

/persons:
  get:
    tags: [Persons]
    summary: Lista pessoas
    parameters:
      - in: query
        name: q
        schema: { type: string }
    responses:
      '200':
        description: Lista de pessoas
        content:
          application/json:
            schema:
              type: array
              items:
                $ref: '#/components/schemas/Person'
  post:
    tags: [Persons]
    summary: Cria pessoa
    requestBody:
      required: true
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/PersonCreateRequest'
    responses:
      '201':
        description: Pessoa criada
        content:
          application/json:
```

```
            schema:
              $ref: '#/components/schemas/Person'

/persons/{id}:
  get:
    tags: [Persons]
    summary: Detalha pessoa
    parameters:
      - $ref: '#/components/parameters/IdPath'
    responses:
      '200':
        description: Pessoa
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/Person'
  patch:
    tags: [Persons]
    summary: Atualiza pessoa
    parameters:
      - $ref: '#/components/parameters/IdPath'
    requestBody:
      required: true
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/PersonUpdateRequest'
    responses:
      '200':
        description: Pessoa atualizada
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/Person'

/persons/{id}/links:
  post:
    tags: [Persons]
    summary: Cria vínculo contextual da pessoa
    parameters:
      - $ref: '#/components/parameters/IdPath'
    requestBody:
      required: true
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/PersonLinkCreateRequest'
    responses:
      '201':
        description: Vínculo criado
```

```yaml
/consents:
  get:
    tags: [Consents]
    summary: Lista consentimentos
    parameters:
      - in: query
        name: person_id
        schema: { type: string, format: uuid }
    responses:
      '200':
        description: Lista de consentimentos
        content:
          application/json:
            schema:
              type: array
              items:
                $ref: '#/components/schemas/Consent'
  post:
    tags: [Consents]
    summary: Registra consentimento
    requestBody:
      required: true
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/ConsentCreateRequest'
    responses:
      '201':
        description: Consentimento registrado
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/Consent'

/consents/{id}/revoke:
  post:
    tags: [Consents]
    summary: Revoga consentimento
    parameters:
      - $ref: '#/components/parameters/IdPath'
    responses:
      '204':
        description: Consentimento revogado

/contacts-capture:
  get:
    tags: [ContactsCapture]
    summary: Lista captações territoriais
    responses:
      '200':
        description: Lista de captações
```

```
        content:
          application/json:
            schema:
              type: array
              items:
                $ref: '#/components/schemas/ContactCapture'
  post:
    tags: [ContactsCapture]
    summary: Cria captação territorial
    requestBody:
      required: true
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/ContactCaptureCreateRequest'
    responses:
      '201':
        description: Captação criada
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/ContactCapture'

/contacts-capture/{id}/classify:
  post:
    tags: [ContactsCapture]
    summary: Classifica ou reclassifica captação
    parameters:
      - $ref: '#/components/parameters/IdPath'
    requestBody:
      required: true
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/ContactCaptureClassifyRequest'
    responses:
      '200':
        description: Captação atualizada
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/ContactCapture'

/contacts-capture/{id}/forward-to-polo:
  post:
    tags: [ContactsCapture]
    summary: Encaminha captação ao polo
    parameters:
      - $ref: '#/components/parameters/IdPath'
    requestBody:
      required: true
```

```
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/ContactCaptureForwardRequest'
    responses:
      '204':
        description: Encaminhamento efetuado

/contacts-capture/{id}/convert-beneficiary:
  post:
    tags: [ContactsCapture]
    summary: Converte captação em beneficiário do polo
    parameters:
      - $ref: '#/components/parameters/IdPath'
    requestBody:
      required: true
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/ContactCaptureConvertRequest'
    responses:
      '201':
        description: Beneficiário criado
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/PoloBeneficiary'

/polos:
  get:
    tags: [Polos]
    summary: Lista polos
    responses:
      '200':
        description: Lista de polos
        content:
          application/json:
            schema:
              type: array
              items:
                $ref: '#/components/schemas/Polo'
  post:
    tags: [Polos]
    summary: Cria polo
    requestBody:
      required: true
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/PoloCreateRequest'
    responses:
```

```
      '201':
        description: Polo criado
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/Polo'

/polos/{id}/beneficiarios:
  get:
    tags: [Polos]
    summary: Lista beneficiários do polo
    parameters:
      - $ref: '#/components/parameters/IdPath'
    responses:
      '200':
        description: Lista de beneficiários
        content:
          application/json:
            schema:
              type: array
              items:
                $ref: '#/components/schemas/PoloBeneficiary'
  post:
    tags: [Polos]
    summary: Vincula beneficiário ao polo
    parameters:
      - $ref: '#/components/parameters/IdPath'
    requestBody:
      required: true
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/PoloBeneficiaryCreateRequest'
    responses:
      '201':
        description: Beneficiário vinculado
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/PoloBeneficiary'

/polos/{id}/frequencias:
  get:
    tags: [Polos]
    summary: Lista frequências do polo
    parameters:
      - $ref: '#/components/parameters/IdPath'
    responses:
      '200':
        description: Lista de frequências
        content:
```

```
          application/json:
            schema:
              type: array
              items:
                $ref: '#/components/schemas/Attendance'
  post:
    tags: [Polos]
    summary: Registra frequência
    parameters:
      - $ref: '#/components/parameters/IdPath'
    requestBody:
      required: true
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/AttendanceCreateRequest'
    responses:
      '201':
        description: Frequência registrada
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/Attendance'

/polos/{id}/ocorrencias:
  get:
    tags: [Polos]
    summary: Lista ocorrências do polo
    parameters:
      - $ref: '#/components/parameters/IdPath'
    responses:
      '200':
        description: Lista de ocorrências
        content:
          application/json:
            schema:
              type: array
              items:
                $ref: '#/components/schemas/Occurrence'
  post:
    tags: [Polos]
    summary: Registra ocorrência
    parameters:
      - $ref: '#/components/parameters/IdPath'
    requestBody:
      required: true
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/OccurrenceCreateRequest'
    responses:
```

```
      '201':
        description: Ocorrência criada
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/Occurrence'

/demands:
  get:
    tags: [Demands]
    summary: Lista demandas
    responses:
      '200':
        description: Lista de demandas
        content:
          application/json:
            schema:
              type: array
              items:
                $ref: '#/components/schemas/Demand'
  post:
    tags: [Demands]
    summary: Cria demanda
    requestBody:
      required: true
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/DemandCreateRequest'
    responses:
      '201':
        description: Demanda criada
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/Demand'

/demands/{id}/assign:
  post:
    tags: [Demands]
    summary: Atribui demanda
    parameters:
      - $ref: '#/components/parameters/IdPath'
    requestBody:
      required: true
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/DemandAssignRequest'
    responses:
      '204':
```

```
        description: Demanda atribuída

/tasks:
  get:
    tags: [Tasks]
    summary: Lista tarefas
    responses:
      '200':
        description: Lista de tarefas
        content:
          application/json:
            schema:
              type: array
              items:
                $ref: '#/components/schemas/Task'
  post:
    tags: [Tasks]
    summary: Cria tarefa
    requestBody:
      required: true
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/TaskCreateRequest'
    responses:
      '201':
        description: Tarefa criada
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/Task'

/tasks/{id}/complete:
  post:
    tags: [Tasks]
    summary: Conclui tarefa
    parameters:
      - $ref: '#/components/parameters/IdPath'
    requestBody:
      required: false
    responses:
      '204':
        description: Tarefa concluída

/events:
  get:
    tags: [Events]
    summary: Lista eventos
    responses:
      '200':
        description: Lista de eventos
```

```
        content:
          application/json:
            schema:
              type: array
              items:
                $ref: '#/components/schemas/Event'
  post:
    tags: [Events]
    summary: Cria evento
    requestBody:
      required: true
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/EventCreateRequest'
    responses:
      '201':
        description: Evento criado
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/Event'

/activities:
  get:
    tags: [Events]
    summary: Lista atividades
    responses:
      '200':
        description: Lista de atividades
        content:
          application/json:
            schema:
              type: array
              items:
                $ref: '#/components/schemas/Activity'
  post:
    tags: [Events]
    summary: Cria atividade
    requestBody:
      required: true
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/ActivityCreateRequest'
    responses:
      '201':
        description: Atividade criada
        content:
          application/json:
            schema:
```

```yaml
              $ref: '#/components/schemas/Activity'

/vereadores/{id}/dashboard:
  get:
    tags: [Dashboards]
    summary: Dashboard executivo do vereador
    parameters:
      - $ref: '#/components/parameters/IdPath'
    responses:
      '200':
        description: Dashboard do vereador
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/VereadorDashboard'

/reports/export:
  post:
    tags: [Reports]
    summary: Gera exportação de relatório
    requestBody:
      required: true
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/ReportExportRequest'
    responses:
      '202':
        description: Exportação solicitada

/audit/logs:
  get:
    tags: [Audit]
    summary: Consulta logs de auditoria
    parameters:
      - in: query
        name: user_id
        schema: { type: string, format: uuid }
    responses:
      '200':
        description: Logs
        content:
          application/json:
            schema:
              type: array
              items:
                $ref: '#/components/schemas/AuditLog'

/privacy/requests:
  get:
    tags: [Privacy]
```

```
      summary: Lista solicitações de privacidade
      responses:
        '200':
          description: Solicitações
          content:
            application/json:
              schema:
                type: array
                items:
                  $ref: '#/components/schemas/PrivacyRequest'
components:
  securitySchemes:
    bearerAuth:
      type: http
      scheme: bearer
      bearerFormat: JWT
  parameters:
    IdPath:
      in: path
      name: id
      required: true
      schema:
        type: string
        format: uuid
    Page:
      in: query
      name: page
      schema:
        type: integer
        default: 1
    PageSize:
      in: query
      name: page_size
      schema:
        type: integer
        default: 20
  schemas:
    ErrorResponse:
      type: object
      required: [detail]
      properties:
        detail:
          type: string
        code:
          type: string
          nullable: true
    LoginRequest:
      type: object
      required: [username, password]
      properties:
        username: { type: string }
```

```
    password: { type: string }
RefreshRequest:
  type: object
  required: [refresh_token]
  properties:
    refresh_token: { type: string }
TokenResponse:
  type: object
  required: [access_token, refresh_token, token_type]
  properties:
    access_token: { type: string }
    refresh_token: { type: string }
    token_type: { type: string, example: Bearer }
    expires_in: { type: integer }
User:
  type: object
  required: [id, username, email, status]
  properties:
    id: { type: string, format: uuid }
    username: { type: string }
    email: { type: string, format: email }
    status: { type: string }
    roles:
      type: array
      items: { type: string }
UserCreateRequest:
  type: object
  required: [username, email, password]
  properties:
    person_id: { type: string, format: uuid, nullable: true }
    username: { type: string }
    email: { type: string, format: email }
    password: { type: string }
UserUpdateRequest:
  type: object
  properties:
    email: { type: string, format: email, nullable: true }
    status: { type: string, nullable: true }
    must_reset_password: { type: boolean, nullable: true }
UserRolesAssignRequest:
  type: object
  required: [role_codes]
  properties:
    role_codes:
      type: array
      items: { type: string }
UserScopesAssignRequest:
  type: object
  required: [scopes]
  properties:
    scopes:
```

```
          type: array
          items:
            type: object
            required: [scope_type, scope_ref_id]
            properties:
              scope_type: { type: string }
              scope_ref_id: { type: string, format: uuid }
    PaginatedUsersResponse:
      type: object
      properties:
        items:
          type: array
          items:
            $ref: '#/components/schemas/User'
        page: { type: integer }
        page_size: { type: integer }
        total: { type: integer }
    Organization:
      type: object
      required: [id, type, name, active]
      properties:
        id: { type: string, format: uuid }
        type: { type: string }
        name: { type: string }
        legal_name: { type: string, nullable: true }
        document_number: { type: string, nullable: true }
        parent_organization_id: { type: string, format: uuid, nullable:
true }
        active: { type: boolean }
    OrganizationCreateRequest:
      type: object
      required: [type, name]
      properties:
        type: { type: string }
        name: { type: string }
        legal_name: { type: string, nullable: true }
        document_number: { type: string, nullable: true }
        parent_organization_id: { type: string, format: uuid, nullable:
true }
    OrganizationUpdateRequest:
      type: object
      properties:
        name: { type: string, nullable: true }
        legal_name: { type: string, nullable: true }
        active: { type: boolean, nullable: true }
    Vereador:
      type: object
      required: [id, person_id, organization_id, active]
      properties:
        id: { type: string, format: uuid }
        person_id: { type: string, format: uuid }
```

```
    organization_id: { type: string, format: uuid }
    active: { type: boolean }
VereadorCreateRequest:
  type: object
  required: [person_id, organization_id]
  properties:
    person_id: { type: string, format: uuid }
    organization_id: { type: string, format: uuid }
Team:
  type: object
  required: [id, organization_id, name, team_type, active]
  properties:
    id: { type: string, format: uuid }
    organization_id: { type: string, format: uuid }
    name: { type: string }
    team_type: { type: string }
    active: { type: boolean }
TeamCreateRequest:
  type: object
  required: [organization_id, name, team_type]
  properties:
    organization_id: { type: string, format: uuid }
    name: { type: string }
    team_type: { type: string }
Person:
  type: object
  required: [id, full_name]
  properties:
    id: { type: string, format: uuid }
    full_name: { type: string }
    social_name: { type: string, nullable: true }
    cpf: { type: string, nullable: true }
    birth_date: { type: string, format: date, nullable: true }
    phone: { type: string, nullable: true }
    secondary_phone: { type: string, nullable: true }
    email: { type: string, format: email, nullable: true }
    gender: { type: string, nullable: true }
PersonCreateRequest:
  type: object
  required: [full_name]
  properties:
    full_name: { type: string }
    social_name: { type: string, nullable: true }
    cpf: { type: string, nullable: true }
    birth_date: { type: string, format: date, nullable: true }
    phone: { type: string, nullable: true }
    secondary_phone: { type: string, nullable: true }
    email: { type: string, format: email, nullable: true }
    gender: { type: string, nullable: true }
    notes: { type: string, nullable: true }
PersonUpdateRequest:
```

```
    type: object
    properties:
      full_name: { type: string, nullable: true }
      social_name: { type: string, nullable: true }
      phone: { type: string, nullable: true }
      email: { type: string, format: email, nullable: true }
      notes: { type: string, nullable: true }
  PersonLinkCreateRequest:
    type: object
    required: [link_type]
    properties:
      organization_id: { type: string, format: uuid, nullable: true }
      vereador_id: { type: string, format: uuid, nullable: true }
      link_type: { type: string }
      metadata:
        type: object
        additionalProperties: true
        nullable: true
  Consent:
    type: object
    required: [id, person_id, consent_type, granted, version]
    properties:
      id: { type: string, format: uuid }
      person_id: { type: string, format: uuid }
      consent_type: { type: string }
      granted: { type: boolean }
      version: { type: string }
      granted_at: { type: string, format: date-time, nullable: true }
      revoked_at: { type: string, format: date-time, nullable: true }
  ConsentCreateRequest:
    type: object
    required: [person_id, consent_type, granted, version]
    properties:
      person_id: { type: string, format: uuid }
      consent_type: { type: string }
      granted: { type: boolean }
      version: { type: string }
      evidence_ref: { type: string, nullable: true }
  ContactCapture:
    type: object
    required: [id, origin, classification, full_name, capture_status]
    properties:
      id: { type: string, format: uuid }
      origin: { type: string }
      classification: { type: string }
      full_name: { type: string }
      phone: { type: string, nullable: true }
      district: { type: string, nullable: true }
      notes: { type: string, nullable: true }
      capture_status: { type: string }
      vereador_id: { type: string, format: uuid, nullable: true }
```

```
    team_id: { type: string, format: uuid, nullable: true }
ContactCaptureCreateRequest:
  type: object
  required: [origin, classification, full_name]
  properties:
    origin: { type: string }
    classification: { type: string }
    full_name: { type: string }
    phone: { type: string, nullable: true }
    district: { type: string, nullable: true }
    notes: { type: string, nullable: true }
    vereador_id: { type: string, format: uuid, nullable: true }
    team_id: { type: string, format: uuid, nullable: true }
    latitude: { type: number, nullable: true }
    longitude: { type: number, nullable: true }
ContactCaptureClassifyRequest:
  type: object
  required: [classification]
  properties:
    classification: { type: string }
    notes: { type: string, nullable: true }
ContactCaptureForwardRequest:
  type: object
  required: [polo_id]
  properties:
    polo_id: { type: string, format: uuid }
    notes: { type: string, nullable: true }
ContactCaptureConvertRequest:
  type: object
  required: [polo_id]
  properties:
    polo_id: { type: string, format: uuid }
    person_id: { type: string, format: uuid, nullable: true }
Polo:
  type: object
  required: [id, organization_id, active]
  properties:
    id: { type: string, format: uuid }
    organization_id: { type: string, format: uuid }
    code: { type: string, nullable: true }
    address_label: { type: string, nullable: true }
    active: { type: boolean }
PoloCreateRequest:
  type: object
  required: [organization_id]
  properties:
    organization_id: { type: string, format: uuid }
    code: { type: string, nullable: true }
    address_label: { type: string, nullable: true }
PoloBeneficiary:
  type: object
```

```
    required: [id, polo_id, person_id, status]
    properties:
      id: { type: string, format: uuid }
      polo_id: { type: string, format: uuid }
      person_id: { type: string, format: uuid }
      source_capture_id: { type: string, format: uuid, nullable: true }
      status: { type: string }
  PoloBeneficiaryCreateRequest:
    type: object
    required: [person_id]
    properties:
      person_id: { type: string, format: uuid }
      source_capture_id: { type: string, format: uuid, nullable: true }
      status: { type: string, nullable: true }
  Attendance:
    type: object
    required: [id, beneficiario_id, activity_date, present]
    properties:
      id: { type: string, format: uuid }
      beneficiario_id: { type: string, format: uuid }
      modalidade_id: { type: string, format: uuid, nullable: true }
      activity_date: { type: string, format: date }
      present: { type: boolean }
      notes: { type: string, nullable: true }
  AttendanceCreateRequest:
    type: object
    required: [beneficiario_id, activity_date, present]
    properties:
      beneficiario_id: { type: string, format: uuid }
      modalidade_id: { type: string, format: uuid, nullable: true }
      activity_date: { type: string, format: date }
      present: { type: boolean }
      notes: { type: string, nullable: true }
  Occurrence:
    type: object
    required: [id, polo_id, severity, title, description, status]
    properties:
      id: { type: string, format: uuid }
      polo_id: { type: string, format: uuid }
      beneficiario_id: { type: string, format: uuid, nullable: true }
      severity: { type: string }
      title: { type: string }
      description: { type: string }
      status: { type: string }
  OccurrenceCreateRequest:
    type: object
    required: [severity, title, description]
    properties:
      beneficiario_id: { type: string, format: uuid, nullable: true }
      severity: { type: string }
      title: { type: string }
```

```
    description: { type: string }
Demand:
  type: object
  required: [id, category, title, priority, status]
  properties:
    id: { type: string, format: uuid }
    person_id: { type: string, format: uuid, nullable: true }
    capture_id: { type: string, format: uuid, nullable: true }
    category: { type: string }
    title: { type: string }
    description: { type: string, nullable: true }
    priority: { type: string }
    status: { type: string }
DemandCreateRequest:
  type: object
  required: [category, title]
  properties:
    person_id: { type: string, format: uuid, nullable: true }
    capture_id: { type: string, format: uuid, nullable: true }
    organization_id: { type: string, format: uuid, nullable: true }
    vereador_id: { type: string, format: uuid, nullable: true }
    category: { type: string }
    title: { type: string }
    description: { type: string, nullable: true }
    priority: { type: string, nullable: true }
    due_at: { type: string, format: date-time, nullable: true }
DemandAssignRequest:
  type: object
  required: [assigned_to_user_id]
  properties:
    assigned_to_user_id: { type: string, format: uuid }
Task:
  type: object
  required: [id, task_type, title, priority, status]
  properties:
    id: { type: string, format: uuid }
    task_type: { type: string }
    title: { type: string }
    description: { type: string, nullable: true }
    priority: { type: string }
    status: { type: string }
    due_at: { type: string, format: date-time, nullable: true }
TaskCreateRequest:
  type: object
  required: [task_type, title]
  properties:
    organization_id: { type: string, format: uuid, nullable: true }
    vereador_id: { type: string, format: uuid, nullable: true }
    polo_id: { type: string, format: uuid, nullable: true }
    person_id: { type: string, format: uuid, nullable: true }
    demand_id: { type: string, format: uuid, nullable: true }
```

```
    assigned_to_user_id: { type: string, format: uuid, nullable: true }
    task_type: { type: string }
    title: { type: string }
    description: { type: string, nullable: true }
    priority: { type: string, nullable: true }
    due_at: { type: string, format: date-time, nullable: true }
Event:
  type: object
  required: [id, title, event_type, start_at, status]
  properties:
    id: { type: string, format: uuid }
    title: { type: string }
    description: { type: string, nullable: true }
    event_type: { type: string }
    status: { type: string }
    start_at: { type: string, format: date-time }
    end_at: { type: string, format: date-time, nullable: true }
EventCreateRequest:
  type: object
  required: [title, event_type, start_at]
  properties:
    organization_id: { type: string, format: uuid, nullable: true }
    vereador_id: { type: string, format: uuid, nullable: true }
    title: { type: string }
    description: { type: string, nullable: true }
    event_type: { type: string }
    start_at: { type: string, format: date-time }
    end_at: { type: string, format: date-time, nullable: true }
Activity:
  type: object
  required: [id, title, activity_type, starts_at]
  properties:
    id: { type: string, format: uuid }
    event_id: { type: string, format: uuid, nullable: true }
    polo_id: { type: string, format: uuid, nullable: true }
    title: { type: string }
    activity_type: { type: string }
    starts_at: { type: string, format: date-time }
    ends_at: { type: string, format: date-time, nullable: true }
    recurrence_rule: { type: string, nullable: true }
ActivityCreateRequest:
  type: object
  required: [title, activity_type, starts_at]
  properties:
    event_id: { type: string, format: uuid, nullable: true }
    polo_id: { type: string, format: uuid, nullable: true }
    organization_id: { type: string, format: uuid, nullable: true }
    title: { type: string }
    activity_type: { type: string }
    starts_at: { type: string, format: date-time }
    ends_at: { type: string, format: date-time, nullable: true }
```

```
    recurrence_rule: { type: string, nullable: true }
VereadorDashboard:
  type: object
  properties:
    vereador_id: { type: string, format: uuid }
    total_captures: { type: integer }
    total_beneficiarios: { type: integer }
    open_demands: { type: integer }
    open_tasks: { type: integer }
    poles:
      type: array
      items:
        type: object
        properties:
          polo_id: { type: string, format: uuid }
          name: { type: string }
          active_beneficiaries: { type: integer }
ReportExportRequest:
  type: object
  required: [report_type]
  properties:
    report_type: { type: string }
    filters:
      type: object
      additionalProperties: true
AuditLog:
  type: object
  properties:
    id: { type: integer }
    user_id: { type: string, format: uuid, nullable: true }
    action: { type: string }
    entity_schema: { type: string }
    entity_name: { type: string }
    entity_id: { type: string, format: uuid, nullable: true }
    created_at: { type: string, format: date-time }
PrivacyRequest:
  type: object
  properties:
    id: { type: string, format: uuid }
    person_id: { type: string, format: uuid }
    request_type: { type: string }
    status: { type: string }
    requested_at: { type: string, format: date-time }
```

---

2) Migrations Alembic por schema

2.1 Estrutura sugerida de versionamento

```
apps/api/alembic/versions/
  0001_create_iam_schema.py
  0002_create_core_schema.py
  0003_create_territory_schema.py
  0004_create_polo_schema.py
  0005_create_events_schema.py
  0006_create_workflow_schema.py
  0007_create_governance_schema.py
  0008_create_analytics_schema.py
```

2.2 `0001_create_iam_schema.py`

```python
"""create iam schema

Revision ID: 0001_create_iam_schema
Revises:
Create Date: 2026-04-10 00:00:01
"""

from alembic import op
import sqlalchemy as sa
from sqlalchemy.dialects import postgresql

revision = "0001_create_iam_schema"
down_revision = None
branch_labels = None
depends_on = None


def upgrade() -> None:
    op.execute("CREATE SCHEMA IF NOT EXISTS iam")

    op.create_table(
        "users",
        sa.Column("id", postgresql.UUID(as_uuid=True), primary_key=True),
        sa.Column("person_id", postgresql.UUID(as_uuid=True), nullable=True),
        sa.Column("username", sa.String(length=120), nullable=False,
unique=True),
        sa.Column("email", sa.String(length=180), nullable=False,
unique=True),
        sa.Column("password_hash", sa.String(length=255), nullable=False),
        sa.Column("status", sa.String(length=20), nullable=False,
server_default="ACTIVE"),
        sa.Column("last_login_at", sa.DateTime(), nullable=True),
```

```
        sa.Column("must_reset_password", sa.Boolean(), nullable=False,
server_default=sa.text("false")),
        sa.Column("created_at", sa.DateTime(), nullable=False,
server_default=sa.text("now()")),
        sa.Column("updated_at", sa.DateTime(), nullable=False,
server_default=sa.text("now()")),
        schema="iam",
    )

    op.create_table(
        "roles",
        sa.Column("id", postgresql.UUID(as_uuid=True), primary_key=True),
        sa.Column("code", sa.String(length=80), nullable=False, unique=True),
        sa.Column("name", sa.String(length=120), nullable=False),
        sa.Column("description", sa.Text(), nullable=True),
        sa.Column("created_at", sa.DateTime(), nullable=False,
server_default=sa.text("now()")),
        schema="iam",
    )

    op.create_table(
        "permissions",
        sa.Column("id", postgresql.UUID(as_uuid=True), primary_key=True),
        sa.Column("code", sa.String(length=120), nullable=False,
unique=True),
        sa.Column("name", sa.String(length=120), nullable=False),
        sa.Column("description", sa.Text(), nullable=True),
        sa.Column("created_at", sa.DateTime(), nullable=False,
server_default=sa.text("now()")),
        schema="iam",
    )

    op.create_table(
        "user_roles",
        sa.Column("id", postgresql.UUID(as_uuid=True), primary_key=True),
        sa.Column("user_id", postgresql.UUID(as_uuid=True),
sa.ForeignKey("iam.users.id", ondelete="CASCADE"), nullable=False),
        sa.Column("role_id", postgresql.UUID(as_uuid=True),
sa.ForeignKey("iam.roles.id", ondelete="CASCADE"), nullable=False),
        sa.Column("is_primary", sa.Boolean(), nullable=False,
server_default=sa.text("false")),
        sa.Column("created_at", sa.DateTime(), nullable=False,
server_default=sa.text("now()")),
        sa.UniqueConstraint("user_id", "role_id",
name="uq_iam_user_roles_user_role"),
        schema="iam",
    )

    op.create_table(
        "role_permissions",
        sa.Column("id", postgresql.UUID(as_uuid=True), primary_key=True),
```

```
        sa.Column("role_id", postgresql.UUID(as_uuid=True),
sa.ForeignKey("iam.roles.id", ondelete="CASCADE"), nullable=False),
        sa.Column("permission_id", postgresql.UUID(as_uuid=True),
sa.ForeignKey("iam.permissions.id", ondelete="CASCADE"), nullable=False),
        sa.Column("created_at", sa.DateTime(), nullable=False,
server_default=sa.text("now()")),
        sa.UniqueConstraint("role_id", "permission_id",
name="uq_iam_role_permissions_role_permission"),
        schema="iam",
    )

    op.create_table(
        "user_scope_assignments",
        sa.Column("id", postgresql.UUID(as_uuid=True), primary_key=True),
        sa.Column("user_id", postgresql.UUID(as_uuid=True),
sa.ForeignKey("iam.users.id", ondelete="CASCADE"), nullable=False),
        sa.Column("scope_type", sa.String(length=30), nullable=False),
        sa.Column("scope_ref_id", postgresql.UUID(as_uuid=True),
nullable=False),
        sa.Column("created_at", sa.DateTime(), nullable=False,
server_default=sa.text("now()")),
        sa.UniqueConstraint("user_id", "scope_type", "scope_ref_id",
name="uq_iam_user_scope_assignments"),
        schema="iam",
    )

    op.create_table(
        "refresh_tokens",
        sa.Column("id", postgresql.UUID(as_uuid=True), primary_key=True),
        sa.Column("user_id", postgresql.UUID(as_uuid=True),
sa.ForeignKey("iam.users.id", ondelete="CASCADE"), nullable=False),
        sa.Column("token_hash", sa.String(length=255), nullable=False),
        sa.Column("expires_at", sa.DateTime(), nullable=False),
        sa.Column("revoked_at", sa.DateTime(), nullable=True),
        sa.Column("created_at", sa.DateTime(), nullable=False,
server_default=sa.text("now()")),
        schema="iam",
    )


def downgrade() -> None:
    op.drop_table("refresh_tokens", schema="iam")
    op.drop_table("user_scope_assignments", schema="iam")
    op.drop_table("role_permissions", schema="iam")
    op.drop_table("user_roles", schema="iam")
    op.drop_table("permissions", schema="iam")
    op.drop_table("roles", schema="iam")
    op.drop_table("users", schema="iam")
    op.execute("DROP SCHEMA IF EXISTS iam CASCADE")
```

2.3 `0002_create_core_schema.py`

```
"""create core schema

Revision ID: 0002_create_core_schema
Revises: 0001_create_iam_schema
Create Date: 2026-04-10 00:00:02
"""

from alembic import op
import sqlalchemy as sa
from sqlalchemy.dialects import postgresql

revision = "0002_create_core_schema"
down_revision = "0001_create_iam_schema"
branch_labels = None
depends_on = None


def upgrade() -> None:
    op.execute("CREATE SCHEMA IF NOT EXISTS core")

    op.create_table(
        "organizations",
        sa.Column("id", postgresql.UUID(as_uuid=True), primary_key=True),
        sa.Column("type", sa.String(length=30), nullable=False),
        sa.Column("name", sa.String(length=255), nullable=False),
        sa.Column("legal_name", sa.String(length=255), nullable=True),
        sa.Column("document_number", sa.String(length=30), nullable=True),
        sa.Column("parent_organization_id", postgresql.UUID(as_uuid=True),
sa.ForeignKey("core.organizations.id"), nullable=True),
        sa.Column("active", sa.Boolean(), nullable=False,
server_default=sa.text("true")),
        sa.Column("created_at", sa.DateTime(), nullable=False,
server_default=sa.text("now()")),
        sa.Column("updated_at", sa.DateTime(), nullable=False,
server_default=sa.text("now()")),
        schema="core",
    )

    op.create_table(
        "persons",
        sa.Column("id", postgresql.UUID(as_uuid=True), primary_key=True),
        sa.Column("full_name", sa.String(length=255), nullable=False),
        sa.Column("social_name", sa.String(length=255), nullable=True),
        sa.Column("cpf", sa.String(length=20), nullable=True),
        sa.Column("birth_date", sa.Date(), nullable=True),
        sa.Column("phone", sa.String(length=30), nullable=True),
        sa.Column("secondary_phone", sa.String(length=30), nullable=True),
        sa.Column("email", sa.String(length=180), nullable=True),
```

```
        sa.Column("gender", sa.String(length=30), nullable=True),
        sa.Column("notes", sa.Text(), nullable=True),
        sa.Column("created_at", sa.DateTime(), nullable=False,
server_default=sa.text("now()")),
        sa.Column("updated_at", sa.DateTime(), nullable=False,
server_default=sa.text("now()")),
        schema="core",
    )

    op.create_table(
        "addresses",
        sa.Column("id", postgresql.UUID(as_uuid=True), primary_key=True),
        sa.Column("person_id", postgresql.UUID(as_uuid=True),
sa.ForeignKey("core.persons.id", ondelete="CASCADE"), nullable=True),
        sa.Column("organization_id", postgresql.UUID(as_uuid=True),
sa.ForeignKey("core.organizations.id", ondelete="CASCADE"), nullable=True),
        sa.Column("label", sa.String(length=60), nullable=True),
        sa.Column("street", sa.String(length=255), nullable=True),
        sa.Column("number", sa.String(length=30), nullable=True),
        sa.Column("complement", sa.String(length=120), nullable=True),
        sa.Column("district", sa.String(length=120), nullable=True),
        sa.Column("city", sa.String(length=120), nullable=True),
        sa.Column("state", sa.String(length=10), nullable=True),
        sa.Column("zip_code", sa.String(length=20), nullable=True),
        sa.Column("latitude", sa.Numeric(10, 7), nullable=True),
        sa.Column("longitude", sa.Numeric(10, 7), nullable=True),
        sa.Column("created_at", sa.DateTime(), nullable=False,
server_default=sa.text("now()")),
        schema="core",
    )

    op.create_check_constraint(
        "ck_core_addresses_owner_xor",
        "addresses",
        "(person_id is not null) <> (organization_id is not null)",
        schema="core",
    )

    op.create_table(
        "vereadores",
        sa.Column("id", postgresql.UUID(as_uuid=True), primary_key=True),
        sa.Column("person_id", postgresql.UUID(as_uuid=True),
sa.ForeignKey("core.persons.id"), nullable=False),
        sa.Column("organization_id", postgresql.UUID(as_uuid=True),
sa.ForeignKey("core.organizations.id"), nullable=False),
        sa.Column("active", sa.Boolean(), nullable=False,
server_default=sa.text("true")),
        sa.Column("created_at", sa.DateTime(), nullable=False,
server_default=sa.text("now()")),
        schema="core",
    )
```

```
    op.create_table(
        "teams",
        sa.Column("id", postgresql.UUID(as_uuid=True), primary_key=True),
        sa.Column("organization_id", postgresql.UUID(as_uuid=True),
sa.ForeignKey("core.organizations.id"), nullable=False),
        sa.Column("name", sa.String(length=255), nullable=False),
        sa.Column("team_type", sa.String(length=30), nullable=False),
        sa.Column("active", sa.Boolean(), nullable=False,
server_default=sa.text("true")),
        sa.Column("created_at", sa.DateTime(), nullable=False,
server_default=sa.text("now()")),
        schema="core",
    )

    op.create_table(
        "team_members",
        sa.Column("id", postgresql.UUID(as_uuid=True), primary_key=True),
        sa.Column("team_id", postgresql.UUID(as_uuid=True),
sa.ForeignKey("core.teams.id", ondelete="CASCADE"), nullable=False),
        sa.Column("person_id", postgresql.UUID(as_uuid=True),
sa.ForeignKey("core.persons.id"), nullable=False),
        sa.Column("user_id", postgresql.UUID(as_uuid=True),
sa.ForeignKey("iam.users.id"), nullable=True),
        sa.Column("function_name", sa.String(length=100), nullable=True),
        sa.Column("active", sa.Boolean(), nullable=False,
server_default=sa.text("true")),
        sa.Column("created_at", sa.DateTime(), nullable=False,
server_default=sa.text("now()")),
        sa.UniqueConstraint("team_id", "person_id",
name="uq_core_team_members_team_person"),
        schema="core",
    )

    op.create_table(
        "person_links",
        sa.Column("id", postgresql.UUID(as_uuid=True), primary_key=True),
        sa.Column("person_id", postgresql.UUID(as_uuid=True),
sa.ForeignKey("core.persons.id"), nullable=False),
        sa.Column("organization_id", postgresql.UUID(as_uuid=True),
sa.ForeignKey("core.organizations.id"), nullable=True),
        sa.Column("vereador_id", postgresql.UUID(as_uuid=True),
sa.ForeignKey("core.vereadores.id"), nullable=True),
        sa.Column("link_type", sa.String(length=50), nullable=False),
        sa.Column("status", sa.String(length=30), nullable=False,
server_default="ACTIVE"),
        sa.Column("start_date", sa.Date(), nullable=True),
        sa.Column("end_date", sa.Date(), nullable=True),
        sa.Column("metadata", postgresql.JSONB(astext_type=sa.Text()),
nullable=True),
        sa.Column("created_at", sa.DateTime(), nullable=False,
```

```
server_default=sa.text("now()")),
        schema="core",
    )

    op.create_table(
        "consents",
        sa.Column("id", postgresql.UUID(as_uuid=True), primary_key=True),
        sa.Column("person_id", postgresql.UUID(as_uuid=True),
sa.ForeignKey("core.persons.id"), nullable=False),
        sa.Column("consent_type", sa.String(length=50), nullable=False),
        sa.Column("granted", sa.Boolean(), nullable=False),
        sa.Column("version", sa.String(length=20), nullable=False),
        sa.Column("granted_at", sa.DateTime(), nullable=True),
        sa.Column("revoked_at", sa.DateTime(), nullable=True),
        sa.Column("captured_by_user_id", postgresql.UUID(as_uuid=True),
sa.ForeignKey("iam.users.id"), nullable=True),
        sa.Column("evidence_ref", sa.Text(), nullable=True),
        sa.Column("created_at", sa.DateTime(), nullable=False,
server_default=sa.text("now()")),
        schema="core",
    )

    op.create_table(
        "attachments",
        sa.Column("id", postgresql.UUID(as_uuid=True), primary_key=True),
        sa.Column("entity_type", sa.String(length=50), nullable=False),
        sa.Column("entity_id", postgresql.UUID(as_uuid=True),
nullable=False),
        sa.Column("file_name", sa.String(length=255), nullable=False),
        sa.Column("mime_type", sa.String(length=100), nullable=True),
        sa.Column("storage_key", sa.Text(), nullable=False),
        sa.Column("uploaded_by_user_id", postgresql.UUID(as_uuid=True),
sa.ForeignKey("iam.users.id"), nullable=True),
        sa.Column("created_at", sa.DateTime(), nullable=False,
server_default=sa.text("now()")),
        schema="core",
    )


def downgrade() -> None:
    op.drop_table("attachments", schema="core")
    op.drop_table("consents", schema="core")
    op.drop_table("person_links", schema="core")
    op.drop_table("team_members", schema="core")
    op.drop_table("teams", schema="core")
    op.drop_table("vereadores", schema="core")
    op.drop_constraint("ck_core_addresses_owner_xor", "addresses",
schema="core", type_="check")
    op.drop_table("addresses", schema="core")
    op.drop_table("persons", schema="core")
```

```
    op.drop_table("organizations", schema="core")
    op.execute("DROP SCHEMA IF EXISTS core CASCADE")
```

2.4 0003_create_territory_schema.py

```
"""create territory schema

Revision ID: 0003_create_territory_schema
Revises: 0002_create_core_schema
Create Date: 2026-04-10 00:00:03
"""

from alembic import op
import sqlalchemy as sa
from sqlalchemy.dialects import postgresql

revision = "0003_create_territory_schema"
down_revision = "0002_create_core_schema"
branch_labels = None
depends_on = None


def upgrade() -> None:
    op.execute("CREATE SCHEMA IF NOT EXISTS territory")

    op.create_table(
        "geo_entities",
        sa.Column("id", postgresql.UUID(as_uuid=True), primary_key=True),
        sa.Column("entity_type", sa.String(length=50), nullable=False),
        sa.Column("entity_id", postgresql.UUID(as_uuid=True),
nullable=False),
        sa.Column("latitude", sa.Numeric(10, 7), nullable=True),
        sa.Column("longitude", sa.Numeric(10, 7), nullable=True),
        sa.Column("geojson", postgresql.JSONB(astext_type=sa.Text()),
nullable=True),
        sa.Column("created_at", sa.DateTime(), nullable=False,
server_default=sa.text("now()")),
        sa.UniqueConstraint("entity_type", "entity_id",
name="uq_territory_geo_entity"),
        schema="territory",
    )

    op.create_table(
        "contacts_capture",
        sa.Column("id", postgresql.UUID(as_uuid=True), primary_key=True),
        sa.Column("captured_by_user_id", postgresql.UUID(as_uuid=True),
sa.ForeignKey("iam.users.id"), nullable=False),
        sa.Column("organization_id", postgresql.UUID(as_uuid=True),
sa.ForeignKey("core.organizations.id"), nullable=True),
        sa.Column("vereador_id", postgresql.UUID(as_uuid=True),
```

```
sa.ForeignKey("core.vereadores.id"), nullable=True),
        sa.Column("team_id", postgresql.UUID(as_uuid=True),
sa.ForeignKey("core.teams.id"), nullable=True),
        sa.Column("person_id", postgresql.UUID(as_uuid=True),
sa.ForeignKey("core.persons.id"), nullable=True),
        sa.Column("origin", sa.String(length=30), nullable=False),
        sa.Column("classification", sa.String(length=30), nullable=False),
        sa.Column("full_name", sa.String(length=255), nullable=False),
        sa.Column("phone", sa.String(length=30), nullable=True),
        sa.Column("district", sa.String(length=120), nullable=True),
        sa.Column("notes", sa.Text(), nullable=True),
        sa.Column("priority_level", sa.String(length=20), nullable=True),
        sa.Column("capture_status", sa.String(length=30), nullable=False,
server_default="NEW"),
        sa.Column("latitude", sa.Numeric(10, 7), nullable=True),
        sa.Column("longitude", sa.Numeric(10, 7), nullable=True),
        sa.Column("created_at", sa.DateTime(), nullable=False,
server_default=sa.text("now()")),
        sa.Column("updated_at", sa.DateTime(), nullable=False,
server_default=sa.text("now()")),
        schema="territory",
    )

    op.create_table(
        "leadership_signals",
        sa.Column("id", postgresql.UUID(as_uuid=True), primary_key=True),
        sa.Column("capture_id", postgresql.UUID(as_uuid=True),
sa.ForeignKey("territory.contacts_capture.id", ondelete="CASCADE"),
nullable=True),
        sa.Column("person_id", postgresql.UUID(as_uuid=True),
sa.ForeignKey("core.persons.id"), nullable=True),
        sa.Column("signal_type", sa.String(length=60), nullable=False),
        sa.Column("role_name", sa.String(length=120), nullable=True),
        sa.Column("organization_name", sa.String(length=120), nullable=True),
        sa.Column("notes", sa.Text(), nullable=True),
        sa.Column("created_by_user_id", postgresql.UUID(as_uuid=True),
sa.ForeignKey("iam.users.id"), nullable=False),
        sa.Column("created_at", sa.DateTime(), nullable=False,
server_default=sa.text("now()")),
        schema="territory",
    )

    op.create_table(
        "demands",
        sa.Column("id", postgresql.UUID(as_uuid=True), primary_key=True),
        sa.Column("person_id", postgresql.UUID(as_uuid=True),
sa.ForeignKey("core.persons.id"), nullable=True),
        sa.Column("capture_id", postgresql.UUID(as_uuid=True),
sa.ForeignKey("territory.contacts_capture.id"), nullable=True),
        sa.Column("organization_id", postgresql.UUID(as_uuid=True),
sa.ForeignKey("core.organizations.id"), nullable=True),
```

```
        sa.Column("vereador_id", postgresql.UUID(as_uuid=True),
sa.ForeignKey("core.vereadores.id"), nullable=True),
        sa.Column("opened_by_user_id", postgresql.UUID(as_uuid=True),
sa.ForeignKey("iam.users.id"), nullable=False),
        sa.Column("assigned_to_user_id", postgresql.UUID(as_uuid=True),
sa.ForeignKey("iam.users.id"), nullable=True),
        sa.Column("category", sa.String(length=60), nullable=False),
        sa.Column("title", sa.String(length=255), nullable=False),
        sa.Column("description", sa.Text(), nullable=True),
        sa.Column("priority", sa.String(length=20), nullable=False,
server_default="MEDIUM"),
        sa.Column("status", sa.String(length=20), nullable=False,
server_default="OPEN"),
        sa.Column("due_at", sa.DateTime(), nullable=True),
        sa.Column("resolved_at", sa.DateTime(), nullable=True),
        sa.Column("resolution_notes", sa.Text(), nullable=True),
        sa.Column("created_at", sa.DateTime(), nullable=False,
server_default=sa.text("now()")),
        sa.Column("updated_at", sa.DateTime(), nullable=False,
server_default=sa.text("now()")),
        schema="territory",
    )

    op.create_table(
        "territorial_actions",
        sa.Column("id", postgresql.UUID(as_uuid=True), primary_key=True),
        sa.Column("organization_id", postgresql.UUID(as_uuid=True),
sa.ForeignKey("core.organizations.id"), nullable=False),
        sa.Column("vereador_id", postgresql.UUID(as_uuid=True),
sa.ForeignKey("core.vereadores.id"), nullable=True),
        sa.Column("team_id", postgresql.UUID(as_uuid=True),
sa.ForeignKey("core.teams.id"), nullable=True),
        sa.Column("created_by_user_id", postgresql.UUID(as_uuid=True),
sa.ForeignKey("iam.users.id"), nullable=False),
        sa.Column("title", sa.String(length=255), nullable=False),
        sa.Column("description", sa.Text(), nullable=True),
        sa.Column("action_type", sa.String(length=40), nullable=False),
        sa.Column("priority", sa.String(length=20), nullable=False,
server_default="MEDIUM"),
        sa.Column("status", sa.String(length=20), nullable=False,
server_default="OPEN"),
        sa.Column("scheduled_at", sa.DateTime(), nullable=True),
        sa.Column("completed_at", sa.DateTime(), nullable=True),
        sa.Column("created_at", sa.DateTime(), nullable=False,
server_default=sa.text("now()")),
        schema="territory",
    )


def downgrade() -> None:
    op.drop_table("territorial_actions", schema="territory")
```

```
    op.drop_table("demands", schema="territory")
    op.drop_table("leadership_signals", schema="territory")
    op.drop_table("contacts_capture", schema="territory")
    op.drop_table("geo_entities", schema="territory")
    op.execute("DROP SCHEMA IF EXISTS territory CASCADE")
```

2.5 Demais migrations

As migrations `0004_create_polo_schema.py` até `0008_create_analytics_schema.py` seguem o mesmo padrão e usam, respectivamente, o DDL final já fechado para:

- `polo.units`, `polo.beneficiarios`, `polo.modalidades`, `polo.matriculas_modalidade`, `polo.frequencias`, `polo.ocorrencias`, `polo.daily_logs`, `polo.purchase_requests`
- `events.events`, `events.activities`, `events.participations`
- `workflow.tasks`, `workflow.task_outcomes`
- `governance.audit_logs`, `governance.access_logs`, `governance.export_logs`, `governance.privacy_requests`
- `analytics.feature_store_operational`, `analytics.mv_vereador_dashboard`

A equipe pode replicar exatamente o formato acima usando a ordem de dependência entre schemas.

---

3) Seeds iniciais de roles, permissions e escopos

3.1 Arquivo sugerido

```
apps/api/scripts/seed_initial_data.py
```

3.2 Conteúdo base

```
import uuid
from dataclasses import dataclass

ROLES = [
    ("ADM_GERAL_REVISA", "Administrador Geral REVISA"),
    ("ADM_REVISA", "Administrador REVISA"),
    ("VEREADOR", "Vereador"),
    ("CHEFE_GABINETE", "Chefe de Gabinete"),
    ("SUPERVISOR_EQUIPE_POLITICA", "Supervisor de Equipe Política"),
    ("ADM_POLO", "Administrador do Polo"),
    ("COORDENADOR_POLO", "Coordenador do Polo"),
    ("COLABORADOR_POLO", "Colaborador do Polo"),
    ("COLABORADOR_GABINETE", "Colaborador de Gabinete"),
    ("COLABORADOR_REVISA", "Colaborador REVISA"),
    ("BENEFICIARIO", "Beneficiário"),
```

```
    ("EMPRESA_PARCEIRA", "Empresa Parceira"),
    ("VOLUNTARIO_AUTOINSCRITO", "Voluntário Autoinscrito"),
]

PERMISSIONS = [
    "auth.login",
    "user.read", "user.create", "user.update", "user.manage_roles",
"user.manage_scopes",
    "organization.read", "organization.create", "organization.update",
    "vereador.read", "vereador.create", "vereador.update",
    "team.read", "team.create", "team.update",
    "person.read", "person.create", "person.update", "person.link",
    "consent.read", "consent.create", "consent.revoke",
    "capture.read", "capture.create", "capture.classify", "capture.forward",
"capture.convert",
    "polo.read", "polo.create", "polo.update", "polo.manage_beneficiary",
    "modality.read", "modality.create", "modality.update",
    "attendance.read", "attendance.create",
    "occurrence.read", "occurrence.create",
    "daily_log.read", "daily_log.create",
    "purchase_request.read", "purchase_request.create",
    "cabinet.read", "cabinet.action.read", "cabinet.action.create",
    "task.read", "task.create", "task.update", "task.complete",
    "demand.read", "demand.create", "demand.update", "demand.assign",
    "event.read", "event.create", "event.update",
    "dashboard.admin.read", "dashboard.vereador.read",
"dashboard.polo.read", "dashboard.cabinet.read",
    "geo.read", "geo.manage",
    "report.read", "report.export",
    "audit.read", "privacy.read", "privacy.process",
]

ROLE_PERMISSIONS = {
    "ADM_GERAL_REVISA": PERMISSIONS,
    "ADM_REVISA": [
        p for p in PERMISSIONS if p not in {"organization.create",
"vereador.create"}
    ],
    "VEREADOR": [
        "cabinet.read", "capture.read", "demand.read", "task.read",
        "event.read", "dashboard.vereador.read", "report.read", "geo.read"
    ],
    "CHEFE_GABINETE": [
        "team.read", "person.read", "person.link", "consent.read",
"capture.read",
        "capture.create", "capture.classify", "capture.forward",
"cabinet.read",
        "cabinet.action.read", "cabinet.action.create", "task.read",
"task.create",
        "task.update", "task.complete", "demand.read", "demand.create",
"demand.update",
```

```
        "demand.assign", "event.read", "event.create", "event.update",
        "dashboard.cabinet.read", "report.read", "geo.read"
    ],
    "SUPERVISOR_EQUIPE_POLITICA": [
        "team.read", "person.read", "capture.read", "capture.create",
"capture.classify",
        "cabinet.read", "cabinet.action.read", "cabinet.action.create",
"task.read",
        "task.create", "task.update", "task.complete", "demand.read",
"demand.create",
        "demand.update", "event.read", "event.create",
"dashboard.cabinet.read", "report.read"
    ],
    "ADM_POLO": [
        "polo.read", "polo.update", "polo.manage_beneficiary",
"modality.read",
        "modality.create", "modality.update", "attendance.read",
"attendance.create",
        "occurrence.read", "occurrence.create", "daily_log.read",
"daily_log.create",
        "purchase_request.read", "purchase_request.create", "task.read",
"task.create",
        "task.update", "task.complete", "demand.read", "demand.create",
"demand.update",
        "event.read", "event.create", "event.update", "dashboard.polo.read",
        "report.read", "geo.read"
    ],
    "COORDENADOR_POLO": [
        "polo.read", "polo.manage_beneficiary", "modality.read",
"modality.create",
        "modality.update", "attendance.read", "attendance.create",
"occurrence.read",
        "occurrence.create", "daily_log.read", "daily_log.create",
"purchase_request.read",
        "purchase_request.create", "task.read", "task.create", "task.update",
        "task.complete", "demand.read", "demand.create", "demand.update",
        "event.read", "event.create", "dashboard.polo.read", "report.read"
    ],
    "COLABORADOR_POLO": [
        "polo.read", "attendance.create", "attendance.read",
"occurrence.create",
        "occurrence.read", "daily_log.create", "daily_log.read",
"task.read", "task.update",
        "task.complete", "demand.read", "demand.create", "event.read"
    ],
    "COLABORADOR_GABINETE": [
        "person.read", "capture.create", "capture.read", "demand.create",
"demand.read",
        "task.read", "task.update", "task.complete", "event.read"
    ],
    "COLABORADOR_REVISA": [
```

```
        "person.read", "person.create", "person.update", "consent.create",
"capture.read",
        "capture.create", "demand.read", "demand.create", "event.read",
"report.read"
    ],
    "BENEFICIARIO": [],
    "EMPRESA_PARCEIRA": ["event.read"],
    "VOLUNTARIO_AUTOINSCRITO": [],
}

SCOPE_TYPES = [
    "GLOBAL",
    "REVISA",
    "VEREADOR",
    "GABINETE",
    "POLO",
    "EQUIPE",
    "SELF",
]

print("Use este módulo como fonte canônica para popular roles, permissions e
matriz role_permissions.")
```

3.3 Seed SQL equivalente

```
insert into iam.roles (id, code, name)
values
  (gen_random_uuid(), 'ADM_GERAL_REVISA', 'Administrador Geral REVISA'),
  (gen_random_uuid(), 'ADM_REVISA', 'Administrador REVISA'),
  (gen_random_uuid(), 'VEREADOR', 'Vereador'),
  (gen_random_uuid(), 'CHEFE_GABINETE', 'Chefe de Gabinete'),
  (gen_random_uuid(), 'SUPERVISOR_EQUIPE_POLITICA', 'Supervisor de Equipe
Política'),
  (gen_random_uuid(), 'ADM_POLO', 'Administrador do Polo'),
  (gen_random_uuid(), 'COORDENADOR_POLO', 'Coordenador do Polo'),
  (gen_random_uuid(), 'COLABORADOR_POLO', 'Colaborador do Polo'),
  (gen_random_uuid(), 'COLABORADOR_GABINETE', 'Colaborador de Gabinete'),
  (gen_random_uuid(), 'COLABORADOR_REVISA', 'Colaborador REVISA'),
  (gen_random_uuid(), 'BENEFICIARIO', 'Beneficiário'),
  (gen_random_uuid(), 'EMPRESA_PARCEIRA', 'Empresa Parceira'),
  (gen_random_uuid(), 'VOLUNTARIO_AUTOINSCRITO', 'Voluntário Autoinscrito');
```

---

4) Blueprint do monorepo com arquivos-base reais

4.1 Estrutura final

```
revisa-platform/
├── README.md
├── .editorconfig
├── .gitignore
├── .env.example
├── docker-compose.yml
├── Makefile
├── docs/
│   └── architecture/
│       └── execution-pack.md
├── apps/
│   ├── api/
│   │   ├── pyproject.toml
│   │   ├── alembic.ini
│   │   ├── Dockerfile
│   │   ├── app/
│   │   │   ├── main.py
│   │   │   ├── core/
│   │   │   ├── api/
│   │   │   ├── domain/
│   │   │   └── shared/
│   │   ├── alembic/
│   │   │   ├── env.py
│   │   │   └── versions/
│   │   └── scripts/
│   ├── web/
│   │   ├── package.json
│   │   ├── next.config.mjs
│   │   ├── tsconfig.json
│   │   ├── Dockerfile
│   │   └── src/
│   └── mobile/
│       ├── pubspec.yaml
│       ├── Dockerfile
│       └── lib/
├── packages/
│   ├── domain-contracts/
│   ├── enums/
│   ├── ui-tokens/
│   └── lint-config/
├── database/
│   ├── ddl/
```

```
│   ├── seeds/
│   └── views/
└── infra/
    ├── docker/
    └── ci/
```

4.2 Arquivos-base reais

`README.md`

```
# REVISA Platform

Monorepo da plataforma REVISA.

## Apps
- `apps/api`: FastAPI + Alembic + PostgreSQL
- `apps/web`: Next.js
- `apps/mobile`: Flutter

## Subida local
```bash
cp .env.example .env
make up
make migrate
make seed
```

```
### `.env.example`

```env
POSTGRES_DB=revisa
POSTGRES_USER=revisa
POSTGRES_PASSWORD=revisa
DATABASE_URL=postgresql+psycopg://revisa:revisa@postgres:5432/revisa
REDIS_URL=redis://redis:6379/0
JWT_SECRET_KEY=change_me
JWT_REFRESH_SECRET_KEY=change_me_too
API_V1_PREFIX=/api/v1
```

`docker-compose.yml`

```
version: "3.9"
services:
  postgres:
    image: postgres:16
    environment:
      POSTGRES_DB: revisa
```

```
      POSTGRES_USER: revisa
      POSTGRES_PASSWORD: revisa
    ports:
      - "5432:5432"
  redis:
    image: redis:7
    ports:
      - "6379:6379"
  api:
    build: ./apps/api
    env_file:
      - .env
    ports:
      - "8000:8000"
    depends_on:
      - postgres
      - redis
  web:
    build: ./apps/web
    ports:
      - "3000:3000"
    depends_on:
      - api
```

`Makefile`

```
up:
    docker compose up --build

down:
    docker compose down

migrate:
    docker compose exec api alembic upgrade head

seed:
    docker compose exec api python scripts/seed_initial_data.py
```

`apps/api/pyproject.toml`

```
[project]
name = "revisa-api"
version = "1.0.0"
description = "REVISA API"
requires-python = ">=3.11"
dependencies = [
  "fastapi>=0.115.0",
  "uvicorn[standard]>=0.30.0",
  "sqlalchemy>=2.0.0",
```

```
  "alembic>=1.13.0",
  "psycopg[binary]>=3.2.0",
  "pydantic>=2.8.0",
  "pydantic-settings>=2.3.0",
  "pyjwt>=2.8.0",
  "passlib[bcrypt]>=1.7.4",
]

[tool.black]
line-length = 100
```

`apps/api/app/main.py`

```
from fastapi import FastAPI
from app.api.v1.api import api_router

app = FastAPI(title="REVISA Platform API", version="1.0.0")
app.include_router(api_router, prefix="/api/v1")

@app.get("/health")
def health() -> dict[str, str]:
    return {"status": "ok"}
```

`apps/api/app/api/v1/api.py`

```
from fastapi import APIRouter
from app.api.v1.routers import auth, users, persons, contacts_capture,
polos, demands, tasks, dashboards

api_router = APIRouter()
api_router.include_router(auth.router, prefix="/auth", tags=["Auth"])
api_router.include_router(users.router, prefix="/users", tags=["Users"])
api_router.include_router(persons.router, prefix="/persons",
tags=["Persons"])
api_router.include_router(contacts_capture.router, prefix="/contacts-
capture", tags=["ContactsCapture"])
api_router.include_router(polos.router, prefix="/polos", tags=["Polos"])
api_router.include_router(demands.router, prefix="/demands",
tags=["Demands"])
api_router.include_router(tasks.router, prefix="/tasks", tags=["Tasks"])
api_router.include_router(dashboards.router, prefix="", tags=["Dashboards"])
```

`apps/api/app/api/v1/routers/auth.py`

```
from fastapi import APIRouter

router = APIRouter()
```

```
@router.post("/login")
def login() -> dict:
    return {
        "access_token": "stub",
        "refresh_token": "stub",
        "token_type": "Bearer",
        "expires_in": 1800,
    }

@router.get("/me")
def me() -> dict:
    return {"id": "stub", "username": "admin"}
```

`apps/api/alembic/env.py`

```
from logging.config import fileConfig
from sqlalchemy import engine_from_config, pool
from alembic import context

config = context.config
if config.config_file_name is not None:
    fileConfig(config.config_file_name)

target_metadata = None


def run_migrations_offline() -> None:
    url = config.get_main_option("sqlalchemy.url")
    context.configure(url=url, literal_binds=True, compare_type=True)
    with context.begin_transaction():
        context.run_migrations()


def run_migrations_online() -> None:
    connectable = engine_from_config(
        config.get_section(config.config_ini_section),
        prefix="sqlalchemy.",
        poolclass=pool.NullPool,
    )
    with connectable.connect() as connection:
        context.configure(connection=connection, compare_type=True)
        with context.begin_transaction():
            context.run_migrations()


if context.is_offline_mode():
    run_migrations_offline()
else:
    run_migrations_online()
```

`apps/web/package.json`

```
{
  "name": "revisa-web",
  "version": "1.0.0",
  "private": true,
  "scripts": {
    "dev": "next dev",
    "build": "next build",
    "start": "next start"
  },
  "dependencies": {
    "next": "14.2.0",
    "react": "18.2.0",
    "react-dom": "18.2.0"
  }
}
```

`apps/web/src/app/page.tsx`

```
export default function Home() {
  return <main>REVISA Platform</main>;
}
```

`apps/mobile/pubspec.yaml`

```
name: revisa_mobile
description: REVISA mobile app
publish_to: 'none'
version: 1.0.0+1

environment:
  sdk: ">=3.4.0 <4.0.0"

dependencies:
  flutter:
    sdk: flutter

flutter:
  uses-material-design: true
```

`apps/mobile/lib/main.dart`

```
import 'package:flutter/material.dart';

void main() {
  runApp(const RevisaApp());
```

```
}

class RevisaApp extends StatelessWidget {
  const RevisaApp({super.key});

  @override
  Widget build(BuildContext context) {
    return const MaterialApp(
      home: Scaffold(
        body: Center(child: Text('REVISA Mobile')),
      ),
    );
  }
}
```

---

5) Ordem prática de uso pela equipe

1. Salvar o bloco do `openapi.yaml` em `packages/domain-contracts/openapi/openapi.yaml`
2. Criar os arquivos Alembic em `apps/api/alembic/versions/`
3. Salvar o seed Python em `apps/api/scripts/seed_initial_data.py`
4. Materializar o blueprint do monorepo conforme a árvore e os arquivos-base
5. Rodar migrations e seed
6. Gerar clients derivados do OpenAPI para web e mobile

---

6) Observações finais

Este pack fecha a base operacional para iniciar execução disciplinada.
O que está mais detalhado e pronto para uso imediato:

- contrato consolidado da API
- padrão de migrations
- catálogo inicial de papéis e permissões
- esqueleto real do repositório

Os próximos artefatos derivados mais naturais são:

- `permissions_seed.sql` completo
- geração do client TypeScript a partir do OpenAPI
- geração do client Dart a partir do OpenAPI
- stubs de routers e services por domínio
- primeiras migrations completas dos schemas restantes